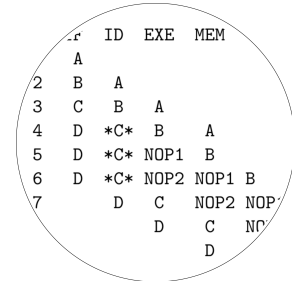


Pipelining Básico

© Juliano Mourão Vieira

28 de setembro de 2026



Sumário

1	Fundamentos	1
1.1	A ideia	3
1.2	Exemplo de execução ideal	3
1.3	Por que funciona?	5
1.4	Mudanças no circuito	6
1.5	Hazards – os problemas inevitáveis	8
1.6	CPIs de processadores não ideais	9
2	Dependências de dados	10
2.1	Stalls	10
2.2	Deteção de dependências	11
2.3	Inserção das bolhas e parada do PC	13
2.4	Forwarding / bypass	14
3	Execução condicional	17
3.1	Diminuindo os stalls no nosso pipeline	17
3.2	Pipelines mais longos	18
3.3	Execução especulativa com branch prediction	19
3.4	Previsão estática	19
3.5	Previsão dinâmica de branches	21

Finalmente chegamos à técnica moderna de projeto de praticamente todos os microprocessadores. Por incrível que pareça, não é difícil de entender.

1 Fundamentos

Vamos dar uma olhada no circuito multiciclo da figura 1, similar ao do livro-texto mas simplificado.

Neste processador, cada instrução será executada em 5 clocks, sem variações, seguindo sequencialmente as operações de:

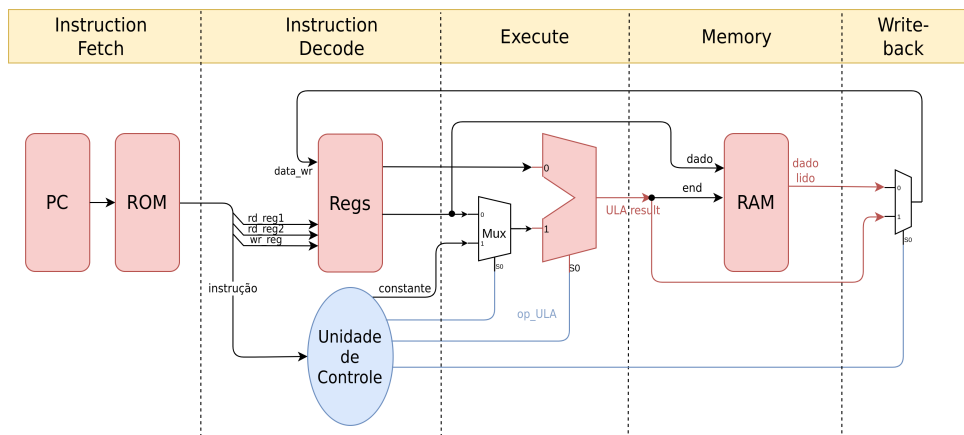


Figura 1: Diagrama em blocos de um microprocessador multiciclo.

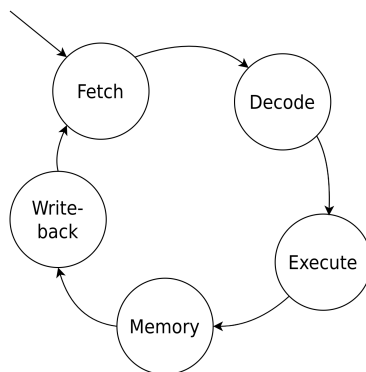


Figura 2: Máquina de 5 estados para nosso microprocessador multiciclo.

1. Instruction fetch, ou busca/leitura da instrução a partir da memória de programa;
2. Instruction decode, ou decodificação/identificação da instrução e geração de sinais de controle;
3. Execute, que neste caso significa meramente fazer a operação da ULA relativa à instrução, se houver;
4. Memory, ou seja, operação de leitura ou de escrita na memória de dados, se houver;
5. Write-back, a escrita final do resultado no banco de registradores.

Podemos ver graficamente a máquina de estados na figura 2. Dependendo da instrução, teremos estados totalmente ociosos (se a instrução não for `lw` nem `sw`, o quarto clock é simplesmente desperdiçado, por ex.) mas não se incomode com isso.

1.1 A ideia

Observe as linhas tracejadas que indicam as partes ativas do circuito em cada clock; perceba que cada um dos estados está utilizando apenas parte do circuito, deixando as outras quatro partes inertes. Podemos tentar aproveitar melhor as partes ociosas, adiantando um pedaço da execução da próxima instrução.

Assim, enquanto estamos fazendo o decode da 1ª instrução, já adiantamos a leitura da 2ª. No clock seguinte, faremos o execute da 1ª e o decode da 2ª, adiantando o fetch da 3ª instrução, e assim por diante.

Uma vantagem óbvia deste esquema parece ser o melhor aproveitamento da circuitaria (e dado o alto consumo de potência estática devido às correntes de fuga, isso já não seria pouca coisa). Mas a vantagem principal que teremos é o aumento de desempenho, a aceleração da execução (explicações detalhadas mais à frente).

A isto se chama **pipelining**, a técnica fundamental de paralelização que consiste em segmentar a execução de todas as instruções para que várias instruções estejam sendo executadas simultaneamente dentro de um processador. Cada segmento é denominado *estágio*.

Há algumas variantes da técnica e muitas, muitas otimizações possíveis, algumas bastante complexas. Ao conjunto de todo este domínio chamamos de ILP – *Instruction Level Parallelism*, ou paralelismo em nível de instrução. Note que isso significa paralelizar a execução *das instruções de um programa*, de forma transparente ao programador e usuário, o que é diferente da paralelização de programas feita pelo programador e executada em diferentes núcleos de microprocessadores. Sendo a ISA a mesma, podemos rodar o mesmo código de máquina em um processador com pipelining ou um processador multiciclo, sem diferença aparente.¹

1.2 Exemplo de execução ideal

Vamos tentar executar um programa simples de assembly RISC-V seguindo nosso esquema de pipeline com 5 estágios, passo-a-passo, bem devagar. Eilo, com as instruções rotuladas por letras:

```
A: addi s1,zero,13
B: add  t3,zero,t4
C: addi s7,zero,-1
D: lw   s2,0(a1)
E: sub  a0,a1,a2
F: add  s4,s4,s4
G: addi s6,zero,0
```

¹Isto de fato acontece. Na arquitetura de microcontroladores Intel MCS51, que já foi muito popular, temos processadores mais recentes com pipelining lado a lado com os processadores multiciclo mais antigos.

No 1º clock, apenas lemos a instrução A da memória de programa.

Ck	IF	ID	EXE	MEM	WB
1	A				

No 2º clock, vamos decodificar A, gerando os sinais de controle devidos, e adiantar a leitura de B.

Ck	IF	ID	EXE	MEM	WB
1	A				
2	B	A			

Como mencionado, no 3º vamos ler C e prosseguir com A e B. Finalmente o processador vai realizar, na ULA neste instante, a soma de zero e 13 como especificado pela instrução A.

Ck	IF	ID	EXE	MEM	WB
1	A				
2	B	A			
3	C	B	A		

No 4º não há nada de interessante (MEM fica ocioso para a instr. A). No clock 5, a seguir, nós temos a *finalização* da instrução A, ou seja, a escrita dos resultados. Note que 5 instruções estão em algum estágio da execução em qualquer dado momento, mas apenas uma delas pode ser finalizada em dado clock neste pipeline.

Ck	IF	ID	EXE	MEM	WB
1	A				
2	B	A			
3	C	B	A		
4	D	C	B	A	
5	E	D	C	B	A

Prosseguindo até o final, percebemos que leva 11 clocks para executar este programa com 7 instruções.

Ck	IF	ID	EXE	MEM	WB
1	A				
2	B	A			
3	C	B	A		
4	D	C	B	A	
5	E	D	C	B	A
6	F	E	D	C	B
7	G	F	E	D	C

8	G	F	E	D
9		G	F	E
10			G	F
11				G

Se fôssemos usar o circuito anterior multiciclo de 5 estados, cada instrução gastaria 5 clocks sem paralelismo; portanto, a execução levaria $7 \times 5 = 35$ clocks.

1.3 Por que funciona?

A ideia principal é que, subdividindo a execução das instruções, podemos “adiantar serviço” paralelizando várias instruções em diferentes estágios do pipeline, mas isso não conta toda a história. O aspecto fundamental da execução ideal em um pipeline é que em todo ciclo de clock temos a finalização de uma instrução.

Desta forma, embora uma instrução em si leve 5 clocks para percorrer todo o pipeline, a cada clock alguma instrução finaliza. Isso nos leva à ideia de **CPI – clocks por instrução**, que indica quantos clocks, *na média, a longo prazo*, são necessários para ter uma instrução finalizada no pipeline.

Para deixar mais claro, suponha que o clock do nosso processador com pipeline de 5 estágios seja 1 GHz. Descontando os 4 clocks iniciais de preenchimento do pipeline, para facilitar, teremos idealmente 1 bilhão de instruções finalizadas em 1 segundo. O CPI ideal de um pipeline portanto é 1,0 clocks por instrução (1×10^9 instr / 1 GHz, ou 1000000000 instruções finalizadas em 1000000000 clocks).

Podemos no entanto adicionar mais estágios, subdividindo ainda mais a execução das tarefas. Se dividirmos cada um dos nossos estágios em dois, teremos 10 estágios e, no caso ideal, poderíamos duplicar o clock para 2 GHz, obtendo o dobro de instruções executadas por segundo na média, a longo prazo.

Um exemplo prático é subdividir o estágio MEM em dois, pois as operações de memória RAM costumam ser bastante demoradas em comparação com as demais. Assim podemos eliminar nosso gargalo e aumentar a frequência de operação.

Um *pipeline profundo* é aquele que possui “muitos” estágios, o que foi uma tendência antes dos multicore, chegando a mais de 30 estágios nos processadores de alto desempenho, com o objetivo fundamental de aumentar o clock. Não há nenhuma definição clara, mas suponho que na prática um processador com mais de 15 estágios pode ser considerado com pipeline profundo.

As desvantagens de termos muitos estágios vão ficar aparentes nas seções posteriores, quando lidarmos com hazards. Ademais, com a tecnologia atual,

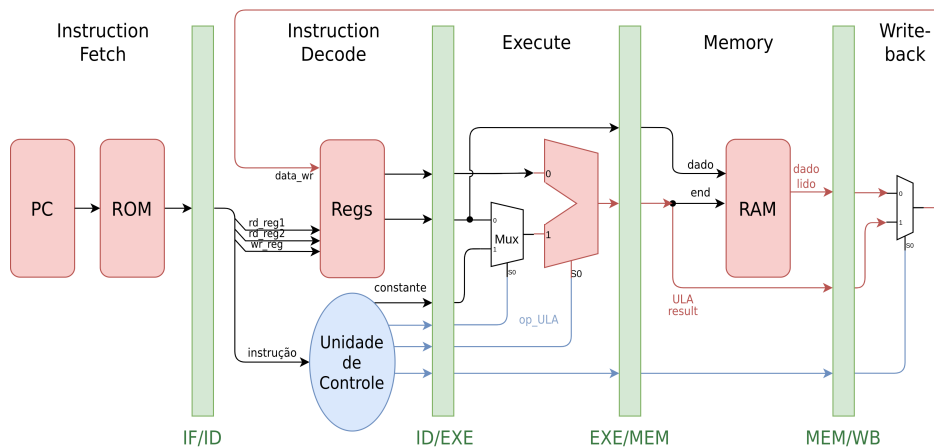


Figura 3: Circuito básico de um pipeline com 5 estágios.

já estamos trabalhando em limites altos de clock e as vantagens de pipelines profundos começam a ficar duvidosas.

1.4 Mudanças no circuito

Uma mudança bastante óbvia é que precisamos guardar as informações produzidas por cada estágio e necessárias para o estágio seguinte, o que implica que precisamos colocar registradores entre os estágios, como podemos ver na figura 3, destacados em verde. (Note-se que vários detalhes estão omitidos no desenho do circuito).

Desta forma, cada instrução em execução terá todos os seus dados armazenados em algum destes registradores comprimidos. O registrador chamado IF/ID irá guardar todas as informações da instrução que foi lida no Instruction Fetch para que ela possa proceder para o Instruction Decode no clock seguinte; de forma análoga, o ID/EXE vai guardar os resultados da decodificação da instrução em dado clock, incluindo valores de registradores, constantes e sinais de controle, ou seja, todas as informações necessárias para que no clock seguinte ela possa efetuar qualquer operação de ULA que precise.

Lembra do 6º clock da nossa simulação anterior?

Ck	IF	ID	EXE	MEM	WB
6	F	E	D	C	B

Se consultarmos o mesmo programa definido anteriormente, podemos visualizar a execução das instruções dentro do pipeline, como ilustrado na figura 4.

Observando na figura a instrução E (`sub a0,a1,a2`, no estágio de Decode), está subentendido que o código de máquina desta instrução (a cons-

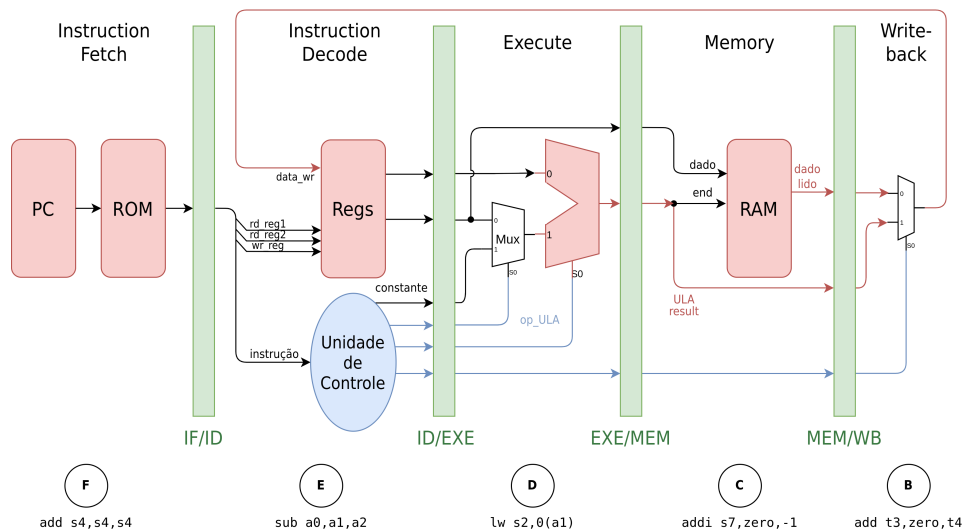


Figura 4: Execução do programa exemplo no pipeline de 5 estágios.

tante hexa 0x40C58533) deve estar guardada em IF/ID pois é necessária agora durante o 6º clock para ser decodificada. Até o final deste clock, os valores de a1 e a2 serão lidos, bem como os sinais de controle serão decodificados (incluindo um op_ULA que indica uma subtração) e estarão prontos para serem escritos em ID/EXE na próxima rampa de subida, compreendendo todos os valores necessários para que no 7º clock a instrução E possa realizar sua operação ($a1 - a2$) na ULA.

Note que pode ser necessário “carregar” várias informações para frente, para uso em estágios posteriores (o que está apenas implícito nos nossos diagramas, sem ligações indicativas). Um exemplo simples é, em instruções que escrevem o resultado no banco de registradores (como add ou addi), o estágio de *Write-back* precisa não só do resultado a escrever, mas do registrador destino, que é determinado pela instrução em si. Este campo de bits é lido no primeiro clock e repassado por todos os estágios, para utilização apenas ao final.

Outro sinal necessário para esta operação é o *write enable* do banco de registradores, que funciona de forma similar ao visto no capítulo de circuitos de um processador ciclo único. Se a instrução vai escrever o resultado, esse “RegWrEn” vai ter que ser ativo em 1; quem vai gerar este sinal é a unidade de controle no estágio 2 (ID), e ele vai ser repassado de clock em clock até sua utilização no estágio 5 (WB). Uma instrução como *sw*, que não escreve no banco, vai ter analogamente RegWrEn em 0, determinado pela UC.

1.5 Hazards – os problemas inevitáveis

Um *hazard* de pipeline é um artefato de execução que atrapalha nosso ideal.

² Essencialmente, trata-se de alguma instrução que não pode ser executada no ciclo de clock previsto por conta de algo que ainda não está disponível a ela. Temos três tipos:

- **Hazard de Dados:** ocorrem quando uma instrução precisa acessar um dado que “ainda não está pronto,” ou seja, cujo resultado ainda não foi escrito no seu destino. Suponha por exemplo a sequência de instruções:

```
A: add s1,s2,s3
B: add t1,t2,s1
```

A instrução A só vai escrever em S2 no 5º clock, mas a instrução B vai tentar ler o valor de S2 no estágio ID, que vai ocorrer *no 3º clock*, antes da escrita do resultado de B em S2. Se executarmos normalmente, ignorando esta dependência de dados, B vai executar incorretamente devido à inconsistência do valor de S2 no circuito.

- **Hazard de Controle:** ocorrem sempre que houver alguma instrução condicional devida a if's ou loops (e existem muitas condicionais nos nossos programas!). A lógica é que uma condicional em assembly é implementada normalmente como um *desvio condicional*, ou seja, uma instrução de branch que escreve um valor de salto no contador de programa, mas só se a condição for verificada. Se esta verificação e escrita ocorrerem apenas no final do pipeline, todas as instruções subsequentes ao branch não podem ser previstas, pois podem ou ser as que seguem o PC ou podem ser as do destino do salto.
- **Hazard Estrutural:** quando duas ou mais instruções precisam do mesmo recurso de hardware, que não é compartilhável. Isso pode ocorrer, por exemplo, quando uma instrução muito lenta ocupa algum estágio por mais de um clock – no processador MIPS original, as divisões de ponto flutuante ocupam exclusivamente a ULA de floats pelas próximas três instruções, que não podem ser contas de floats. Outro exemplo é tentar fazer um pipeline com ambas as memórias de dados e de programa no mesmo CI, provocando uma suspensão temporária na leitura de instruções sempre que houver uma instrução lw ou sw de acesso a dados sendo executada.

²*Hazard* costuma ser traduzido como “risco” ou “perigo”, mas creio que em nosso contexto a palavra “empecilho” seria melhor, ou hazard mesmo, em inglês.

Hazards estruturais são raros e vamos deixá-los de lado. Os outros dois tipos, no entanto, são praticamente inevitáveis e podem impactar bastante no desempenho.

Temos três soluções evidentes:

- **Ignorar** o problema. Obviamente isso vai gerar uma execução incorreta, e não queremos isso. Surpreendentemente, em alguns raros casos apenas, é isso o que é feito, cabendo ao compilador ou programador assembly evitar conscientemente os casos proibidos.³
- **Esperar** a situação se resolver, ou seja, deixar a instrução problemática avançar no pipeline segurando todas as outras posteriores. Quando essa instrução for finalizada, o hazard deixa de existir e podemos, então, prosseguir normalmente.
- **Chunchar**, claro. Perdão, criar circuitos de contorno para otimização, utilizando técnicas às vezes complexas que serão vistas em ILP avançada.

A solução trivial é esperar a finalização da instrução problemática, travando todas as instruções posteriores no pipeline, sem se mover, até que a informação necessária seja escrita. Os ciclos perdidos com instruções que não avançam são chamados de *stalls* (enrolação ou parada, em bom português).

1.6 CPIs de processadores não ideais

É importante notar que, assumindo que os *stalls* são nossa técnica trivial de resolver hazards, nós vamos perder clocks durante a execução, ou seja, entre a instrução problemática e a seguinte, teremos vários clocks sem finalizar instrução nenhuma.

Isso tem uma consequência imediata resultando na piora do nosso CPI, porque vamos gastar mais clocks para executar 100 instruções com hazards do que para executar 100 instruções sem hazards. Um CPI pior, portanto, é *maior* do que 1,0 (gasta *mais* clocks por instrução, na média).

$$CPI_{ideal} = 1,0$$

$$CPI_{real} > 1,0$$

A maioria dos nossos chunch... das nossas técnicas de ILP avançada são destinados a minimizar ou até mesmo eliminar esses e outros stalls, com o objetivo claro de melhorar a CPI através do melhor aproveitamento dos clocks pelo circuito.

³Um exemplo é a divisão em float do MIPS, mencionada logo anteriormente como hazard estrutural.

Se quanto mais baixo o CPI, melhor para o desempenho, qual será o limite? E aqui fica uma deixa para depois: uma $CPI < 1,0$ é possível, com processadores superescalares.

2 Dependências de dados

Observe o programa a seguir:

```
A: addi s1,zero,13
B: addi s7,zero,-1
C: add s2,s1,s7
D: lw a1,0(a2)
```

Vamos iniciar a execução dele numa simulação:

Ck	IF	ID	EXE	MEM	WB
1	A				
2	B	A			
3	C	B	A		
4	D	*C*	B	A	

Perceba que no 4º clock temos um impasse: a instrução C tem que ler os valores de s1 e s7 do banco de registradores, mas o banco *ainda não foi atualizado* com os resultados das instruções A e B, ou seja, está com valores inconsistentes ao momento de execução de C. Se ignorarmos esse hazard, a instrução C vai produzir um valor incorreto.

2.1 Stalls

Como mencionado, o modo mais fácil de resolver esta dependência de dados é fazer a instrução problemática esperar até que as anteriores sejam finalizadas, garantindo o valor correto nos registradores. Para isso, vamos incluir uma instrução NOP (*No OPeration*, não faça nada ⁴), denominada como *bolha*, no pipeline e vamos paralisar a execução das instruções posteriores. Portanto o próximo clock ficará assim:

Ck	IF	ID	EXE	MEM	WB
1	A				
2	B	A			
3	C	B	A		
4	D	*C*	B	A	
5	D	*C*	NOP1	B	A

⁴No RISC-V, NOP é uma pseudoinstrução correspondente à instrução nativa `addi zero,zero,0`, que soma o valor 0 ao registrador zero, que ademais não permite escritas. O seu código de máquina é 0x0013.

Como a instrução A está sendo finalizada, o valor definitivo de s1 está disponível para C, resolvendo esta dependência de dados. Mas ainda temos a dependência de s7 que será escrito por B apenas no clock seguinte, então precisaremos inserir mais um stall no pipeline, como no diagrama:

Ck	IF	ID	EXE	MEM	WB
1	A				
2	B	A			
3	C	B	A		
4	D	*C*	B	A	
5	D	*C*	NOP1	B	
6	D	*C*	NOP2	NOP1	B

Agora sim a dependência de dados entre C e B está resolvida, já que B está sendo finalizada no estágio de Write-back, e a execução pode prosseguir normalmente:

Ck	IF	ID	EXE	MEM	WB
1	A				
2	B	A			
3	C	B	A		
4	D	*C*	B	A	
5	D	*C*	NOP1	B	
6	D	*C*	NOP2	NOP1	B
7		D	C	NOP2	NOP1
8			D	C	NOP2
9				D	C
10					D

Note, enfim, que não é possível detectar as dependências quando a instrução C ainda está no 1º estágio, de Instruction Fetch, porque ela *ainda está sendo lida*; somente depois da sua leitura, com ela guardada no registrador IF/ID, é que ela poderá ser examinada.

2.2 Detecção de dependências

Para detectarmos uma dependência de dados, basta verificar se alguma instrução presente no pipeline, ainda não finalizada, escreve em um registrador que é lido por outra instrução posterior no pipeline (escritas em memória e hazards de controle são tratados à parte, como veremos). Como os registradores são codificados sempre nos mesmos bits nas instruções RISC-V, torna-se quase trivial sua detecção, necessitando apenas que o código de máquina da instrução acompanhe ela durante o percurso no pipeline.

Detecção da leitura dos registradores

Vamos sempre detectar dependências já no 2º estágio, que é onde poderemos tomar alguma medida; para tanto, necessitamos identificar primeiramente o tipo da instrução presente em IF/ID. Os formatos R, I, S e B trabalham com um registrador fonte *rs1* em `instr[19:15]` e os formatos R, S e B também utilizam um segundo registrador fonte *rs2* identificado pelos bits `instr[24:20]`. Instruções U e J não usam registradores fonte.

Para determinar os formatos de interesse, restringindo-nos ao subset RV32I, podemos consultar a tabela de instruções ⁵ e observar os padrões, levantando equações lógicas. Por exemplo, observamos que os formatos U e J codificam apenas 3 instruções: `jal`, `lui` e `auipc`. Portanto podemos detectar o uso ou não de *rs1* pela instrução com:

```
usa_rs1(opcode) = NOT (instr_jal OR instr_lui OR instr_auipc)
```

A identificação da instrução é direta, apenas decodificando seu opcode nos 6 bits LSB da instrução, como visto no circuito ciclo único anteriormente. Para sabermos se a instrução utiliza *rs2*, podemos proceder de forma similar, detectando se a instrução é I, ou seja, se o opcode presente em IF/ID é `11100112` ou `00100112` ou `00000112` ou `11001112` (respectivamente, instruções `ecall` ou `ebreak`, de ULA com imediato, de leitura de memória ou `jalr`). Se for I ou U ou J, não se usa *rs2*.

Detecção da escrita

As instruções que *não escrevem* um resultado em um registrador são poucas: escritas em memória (opcode `01000112`) branches (que fecham o opcode `11000112`), `ecall` e `ebreak` (ambas com opcode `11100112`). ⁶ Portanto, para sabermos se uma instrução escreve algo em um registrador, usamos:

```
escreve_reg(opcode) = NOT (opcode=01000112 OR opcode=11000112  
                           OR opcode=11100112)
```

Note que precisamos comparar os opcodes de todos os estágios posteriores, portanto precisamos dos códigos de máquina das instruções guardadas tanto em ID/EXE como EXE/MEM (vamos considerar a instrução em MEM/WB como tendo “resultado acessível neste mesmo clock”, não configurando hazard em si).

⁵Por exemplo, em <https://github.com/jameslzh/riscv-card/blob/master/riscv-card.pdf>.

⁶Também é necessário verificar se o registrador destino não é a constante zero (o registrador `x0`), mas estamos omitindo aqui esta technicalidade para simplificar.

Comparação, afinal [OPCIONAL]

A comparação em si é trivial, pois a igualdade pode ser implementada como uma operação XOR bit-a-bit e uma simples porta AND entre todos os XORs realizados (confira). Temos então as equações lógicas:

```
opcID = instr_IF_ID[6:0]

rs1ID = instr_IF_ID[19:15]

rs2ID = instr_IF_ID[24:20]

opcEXE = instr_ID_EXE[6:0]

rdestEXE = instr_ID_EXE[11:7]

opcMEM = instr_EXE_MEM[6:0]

rdestMEM = instr_EXE_MEM[11:7]

dep_ID_EXE = escreve_reg(opcEXE) AND (
    (usa_rs1(opcID) AND rdestEXE=rs1ID) OR
    (usa_rs2(opcID) AND rdestEXE=rs2ID) )

dep_ID_MEM = escreve_reg(opcMEM) AND (
    (usa_rs1(opcID) AND rdestMEM=rs1ID) OR
    (usa_rs2(opcID) AND rdestMEM=rs2ID) )

dep_detectada = dep_ID_EXE OR dep_ID_MEM,
```

onde `instr_ID_EXE` é o código de máquina guardado no 2º estágio (os bits LSB 6 a 0 da instrução) e assim por diante.

2.3 Inserção das bolhas e parada do PC

Se quisermos usar o método trivial de resolução de hazards, ou seja, o *stall*, precisamos saber como inserir as bolhas e parar o PC, mas essas ações são na verdade muito simples.

Uma bolha é apenas uma instrução NOP que é inserida no pipeline no 3º estágio (no nosso caso). Os demais sinais guardados no registrador ID/EXE podem ser zerados facilmente com um reset dele, o que pode ser importante para os sinais gerados pela unidade de controle: ao zerarmos eles, pela convenção, estamos desativando qualquer operação posterior possível.

A pseudoinstrução NOP na realidade é traduzida para a instrução nativa (uma instrução sem efeito nenhum), que gera um código de máquina de

32 bits 0x00000013 – portanto se escrevermos este número no campo de código de máquina da instrução, guardado em ID/EXE, estamos garantindo a ociosidade em EXE, que será propagada até o final.

Por fim, a paralisação do PC (e do ID!) é mais fácil do que parece: basta condicionar o *WriteEnable* do PC a não detectarmos nenhuma dependência; quando houver dependência de dados no pipeline, o *PC_wr_en* vai a zero e o valor do PC simplesmente não avança para o próximo endereço. De forma similar, o ID/EXE guarda a instrução lida no clock anterior e iremos desabilitar sua escrita desativando seu *write_enable*, de forma que ele não vai armazenar a instrução lida no estágio IF.

2.4 Forwarding / bypass

Existe uma forma mais otimizada para resolver os hazards de dados, baseada na ideia de que as informações que precisamos já estão prontas dentro do pipeline, apenas ainda não foram escritas (o que só ocorre na finalização no estágio WB). Então vamos adiantar estes resultados (fazer um forwarding) com um atalho (bypass) no caminho de dados padrão.

Funcionamento

Veja o programa a seguir:

```
A: lw    s4,0(t4)
B: addi  s2,zero,-13
C: addi  s3,zero,51
D: add   s1,s2,s3
E: add   s4,s4,s4
```

Observe que D depende tanto de B como de C, mas os resultados de B e C (simples somas) já estão determinados pelo pipeline, mesmo com as instruções ainda não finalizadas. Podemos ver isso ilustrado na figura 5, com as dependências das instruções circuladas e com a indicação em amarelo dos resultados das instruções B e C já calculados.⁷

A ideia, muito simples, é utilizar os dados que desejamos ao invés dos dados lidos do banco de registradores, que seriam incorretos. Para tanto, vamos apenas colocar um multiplexador para selecionar qual o dado que realmente desejamos para ser gravado em ID/EXE, como visto na figura 6.

Implementação

A forma de detectar dependências segue o que foi apresentado na seção 2.2, ou seja, basta compararmos o registrador destino no campo *rd* das instruções

⁷Numa implementação real, os valores seriam obtidos dos registros interestágios (EXE/MEM e MEM/WB) e “forwardados” para o estágio EXE, logo antes da ULA; neste texto, adotei uma simplificação por questões didáticas.

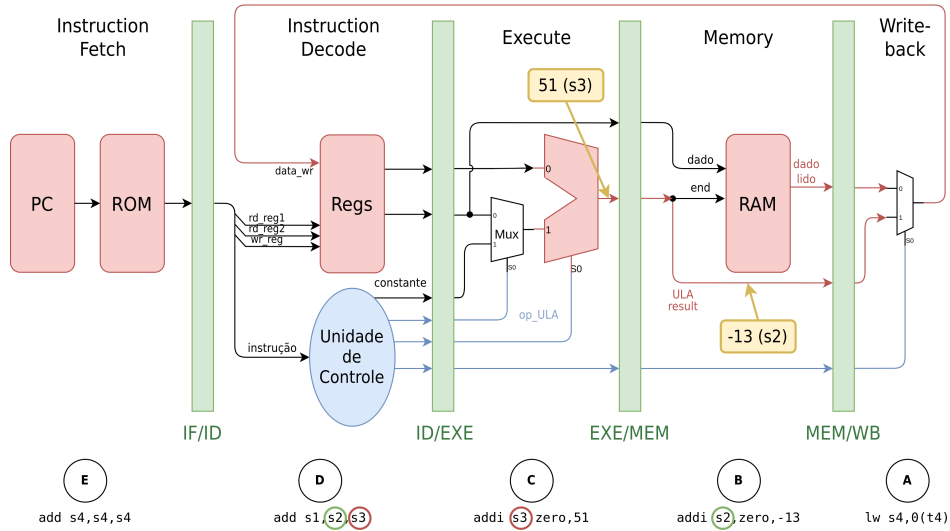


Figura 5: Pipeline mostrando dados com dependência posterior já calculados.

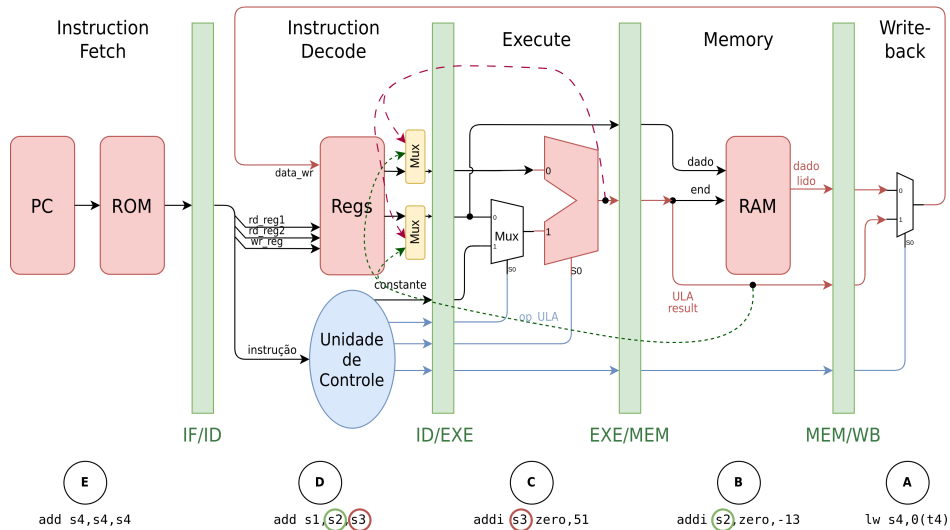


Figura 6: Circuito com Muxes para bypass, com ligações dos dados de forwarding em tracejado.

nos estágios EXE e MEM (e WB, dependendo de como implementarmos) com os dois registradores fonte dos campos rs1 e rs2 da instrução em ID, levando em conta os formatos de instrução para determinar se rs1 e/ou rs2 são utilizados pela instrução em ID.

De acordo com esta informação, podemos fazer a seleção da fonte adequada na entrada dos valores do registrador ID/EXE. Se não houver dependência nenhuma para fazer bypass, simplesmente vamos selecionar o valor lido do banco de registradores; caso haja, vamos selecionar o resultado da operação direto do estágio de interesse.

Ou seja, trata-se de um simples multiplexador, com a seleção dele determinada pelo circuito de detecção de dependências. A única coisa que pode ser um pouco confusa é que temos três estágios possíveis, mas isso pode ser resolvido facilmente com um codificador de prioridade de três fontes (ou um bom e velho Karnaugh, para os mais nostálgicos).⁸

Limitações

Observe uma sequência de duas instruções:

```
...  
C: lw  a1,0(a2)  
D: add a3,zero,a1  
...
```

Há uma dependência de dados entre C e D devido ao registrador `a1`. Se o pipeline estiver como `[E *D* C B A]`, a instrução D não pode prosseguir sem o dado da instrução `lw` em C; no entanto, C só vai para o estágio MEM no próximo clock, ou seja, o dado *ainda não foi lido...* Neste caso, no nosso microprocessador, *é inevitável a inserção de um stall*. Portanto toda vez que houver uma leitura de memória, o uso deste dado pela instrução imediatamente seguinte vai causar uma perda de clock e prejudicar nossa CPI.⁹ Um bom compilador irá tentar reordenar as instruções geradas, adiantando alguma operação que não depende destas, para evitar essa bolha. Técnicas mais avançadas de processadores também podem tentar evitar estas perdas.

Por fim, há o caso de pipelines profundos. No nosso exemplo, precisamos fazer o forwarding de dois, ou talvez três, estágios posteriores, para dois registradores fonte possíveis lidos do banco de registradores, com 32 bits cada. Isto nos dá multiplexadores com um total de $2 \times 2 \times 32 = 128$ fios de entrada, o que não é tanto assim, e um número de portas lógicas proporcional a eles.

⁸*Sugestão ao leitor:* mostre como implementar este circuito, lembrando ou pesquisando codificadores de prioridades.

⁹A exceção é caso a instrução seguinte ao `lw` for uma escrita com `sw`, mas precisamos de mais circuitaria para o bypass deste caso específico.

No entanto, suponha que estejamos trabalhando com registradores de 64 bits e que tenhamos 20 estágios para talvez fazermos o bypass; isso nos daria um total de $2 \times 20 \times 64 = 1280$ fios com tantas portas proporcionais a isso. Este número é bastante alto e gera diversos problemas no projeto. A consequência é que temos que utilizar alguma outra técnica ou então não fazemos o forwarding de dados de todos os estágios potenciais, selecionando parte dos estágios para isso e utilizando-nos de bolhas para os demais, simplificando o projeto mas pagando com stalls por isso.

3 Execução condicional

Hazards de controle não possuem nenhuma solução evidente para serem remediados: mesmo a primeira sugestão, na subseção a seguir, não é universalmente aplicável porque a busca de instruções em pipelines profundos toma vários clocks e pode ser bastante não-linear (vamos ver isso de perto no diagrama em blocos da Intel).

O livro-texto cita uma avaliação da quantidade de condicionais executadas por um processador em uma “situação típica” e estima que 34% das instruções executadas em um RISC-V são branches.¹⁰ Se espera-se que 1/3 das instruções sejam condicionais, cada clock perdido em uma condicional impacta muito o desempenho.

Por exemplo, suponha como uma ilustração pessimista que perdêssemos 4 clocks em 50% dos branches no pipeline; isso quer dizer que a cada 100 instruções, teremos perdas de 4 clocks em metade dos 34 branches. Isso dá 17 instruções das 100 originais que estão perdendo 4 clocks, reduzindo nossa CPI. Ao invés de gastarmos os cerca de 100 clocks num pipeline ideal, gastaríamos $100 + 4 \cdot 17 = 168$ clocks.

Portanto é crucial para os sistemas de alto desempenho que as técnicas implementadas na microarquitetura tenham bom resultado.

3.1 Diminuindo os stalls no nosso pipeline

No nosso circuito de 5 estágios, podemos tomar uma atitude muito simples que reduz perdas de clock: vamos adiantar as comparações de branches condicionais para o segundo estágio. Precisamos de um pouco de circuito extra, mas agora temos apenas um clock em jogo que talvez seja desperdiçado, correspondente à instrução seguinte ao branch.

Neste caso, sempre lemos a instrução seguinte, supondo que o branch não vai ocorrer e que o PC segue seu fluxo normal. Mas caso o branch ocorra,

¹⁰Cabem aqui uma explicação ou duas. Os programas executados são relativos ao *benchmark* SPECint CPU2006 para testar desempenho de números inteiros, que forçam tarefas algorítmicas como o gcc, compactação de dados, compressão de vídeo e outros. A quantidade de branches é medida durante a execução de “testes padrão” destes aplicativos, ou seja, sem casos extremos.

nós precisamos “retirar” a instrução do pipeline de alguma forma. Veja um programa exemplo:

```
...
A: add a0,zero,zero
B: add t1,zero,0
C: beq s1,s2,X
D: lw  t2,0(t1) # Esta instrução será lida com certeza
E: add t2,t2,s2
...
X: addi t1,t1,0x123
Y: lw  t2,0(t1)
...
```

Agora nós temos dois caminhos de execução; considerando a representação das instruções no pipeline como [IF ID EXE MEM WB] identificando os estágios:

- Caso $s1 \neq s2$, o pipeline vai ficar com [E D C B A], pois não haverá o salto de C para X.
- Caso $s1 == s2$, o pipeline vai ficar com [X D C B A], pois haverá o salto de C para X.

O primeiro caso não precisa de ajuste nenhum, já que a instrução D deve ser mesmo executada, mas no segundo caso ela está ali indevidamente: basta olhar para o código para ver que o programa deveria executar [Y X C B A], sem D.

Não podemos de fato “retirar” a instrução D do pipeline com facilidade (o pipeline é basicamente uma FIFO, inflexível) mas podemos sim transformar ela em uma bolha, de forma similar ao que foi feito nos hazards de dados, zerando todos os bits de controle e escrevendo NOP (0x00000013) no código da instrução.

Portanto, se $s1 == s2$, teremos que escrever o endereço de X no PC (ao invés de PC+4, o endereço de E) e ao mesmo tempo gerar um sinal de controle de `clear` para o estágio ID (caso $s1 \neq s2$, o PC continua normalmente e o `clear` fica desativado). O resultado final é que teremos no pipeline [X NOP C B A], perdendo um clock mas evitando a execução indesejável da instrução D.

3.2 Pipelines mais longos

A melhoria anterior, de adiantar a comparação, funciona bem no nosso pipeline: toda vez em que houver um desvio condicional tomado (condição verdadeira no branch), perdemos um clock, e quando o desvio não for tomado,

não perdemos clock nenhum. Ainda assim teríamos uma perda perceptível de clocks.

No caso de processadores de alto desempenho, com pipelines mais longos, a busca da instrução leva vários clocks e a nossa capacidade de determinar se o salto deve ou não ocorrer fica inviabilizada até praticamente o final do pipeline, e não podemos adiantar isso. Mesmo que seja possível adiantar para logo depois de finalizar o fetch, se tivermos que gastar 10 clocks a cada branch tomado (e considerando a quantidade de condicionais executadas), o impacto no desempenho seria catastrófico.

Portanto vamos necessitar de alguma técnica mais avançada do que “se o branch saltar, imponha um stall logo atrás dele.”

Mas podemos caracterizar melhor nossa ideia anterior, sugerindo novos caminhos.

3.3 Execução especulativa com branch prediction

Podemos entender a instrução seguinte ao branch como sendo “executada especulativamente” por alguns estágios até que seja possível decidir se o salto ocorreu ou não, prosseguindo normalmente caso não tenha ocorrido salto ou “retirando-a” do pipeline caso tenha ocorrido.

Esta ideia, que também é aplicada em outras instâncias (como na leitura de memória), pode ser vista como um chute sobre uma situação incerta, para adiantar trabalho dentro do pipeline – se tivermos que esperar a situação se resolver sem prosseguir adiante, com stalls, perderíamos os clocks com 100% de certeza, então parece uma boa ideia especular sobre a tal situação incerta. O procedimento é: vamos especular sobre o que é mais provável acontecer e prosseguir como se tivesse acontecido; quando a situação se resolve, verificamos se nossa especulação foi correta e, caso tenha sido, nada mais é necessário. Infelizmente, caso tenhamos especulado de forma incorreta, precisamos reverter tudo o que foi feito e reexecutar o programa com o chute agora corrigido.

Note que certas situações, como branches condicionais, são impossíveis de se prever corretamente com certeza absoluta sem utilizar uma máquina do tempo. Portanto vamos *tentar prever* o resultado da condicional baseado em algum critério, e executar especulativamente as instruções que seguem. Por isso as técnicas que veremos aqui se chamam *branch prediction*.

3.4 Previsão estática

Podemos fazer uma previsão simples sobre a condicional que desejamos especular:

- *Se o destino do salto é para trás, o salto provavelmente deve ocorrer.* Afinal, isso (quase) certamente é um loop!

- *Se o destino do salto é para frente, o salto provavelmente não deve ocorrer.* A saída de alguns loops pode ser implementada assim, além de condicionais normais.

Estas decisões favorecem tremendamente os loops que os compiladores tipicamente implementam: os *whiles* fazem a comparação *antes* do loop e vão saltar para frente apenas uma vez na saída, e não saltar tantas vezes quantas forem as iterações; e os *do-whiles* ou *repeat-untils* vão usualmente saltar para trás nas comparações, tantas vezes quantas forem as iterações, e irão não saltar apenas uma vez na saída.

Para entender a importância destas ideias, pense na quantidade de loops que um programa comum costuma ter, e o percentual de tempo que ele vai gastar nestes loops. Uma simples repetição de 100 vezes para zerar um vetor vai demorar, proporcionalmente, tanto quanto cem linhas de código! Pelo fato de loops serem muito comuns e de serem executados por dezenas, centenas, milhares de iterações, podemos supor que, na maior parte do tempo, os processadores estarão executando o código de algum loop. Portanto a previsão de branches para execução especulativa é crucial em processadores de alto desempenho, e nossos critérios parecem ser bastante razoáveis para melhorar nossa adivinhação do futuro do processador, já que não podemos dispor de um tarólogo competente implementado em silício.

Resta explicar o que ocorre quando a condição do branch ficar clara para o microprocessador. Como mencionado, quando a comparação for resolvida poderemos comparar com nossa previsão e fazer os ajustes necessários, e se tivermos acertado, nenhum ajuste é exigido. Mas se tivermos errado, precisamos desfazer tudo o que foi realizado especulativamente.

O nome do procedimento que faremos é *pipeline flush*, que consiste simplesmente em limpar todo o pipeline das instruções armazenadas nele e recomençar, lá do instruction fetch, no verdadeiro endereço destino da condicional executada. Isso implica em perdermos os clocks de todo o pipeline (e.g. se tivermos 15 estágios, perderemos 15 clocks), pois vamos limpar tudo; portanto, quanto mais estágios tiver o pipeline, pior serão nossas consequências do erro de previsão.

Para implementar o flush (sim, é a descarga da privada do banheiro, faça as piadas, não tem por que perder a chance), basta zerarmos todos os sinais de controle em todos os registradores interestágios, já que pela convenção um sinal está inativo quando em zero, o que é simplesmente um *reset* dos registradores, colocando todos os bits em zero. Assim, todas as escritas e ações não irão ocorrer. Dependendo da implementação, pode ou não também ser necessário colocar explicitamente uma instrução NOP nos estágios do pipeline.

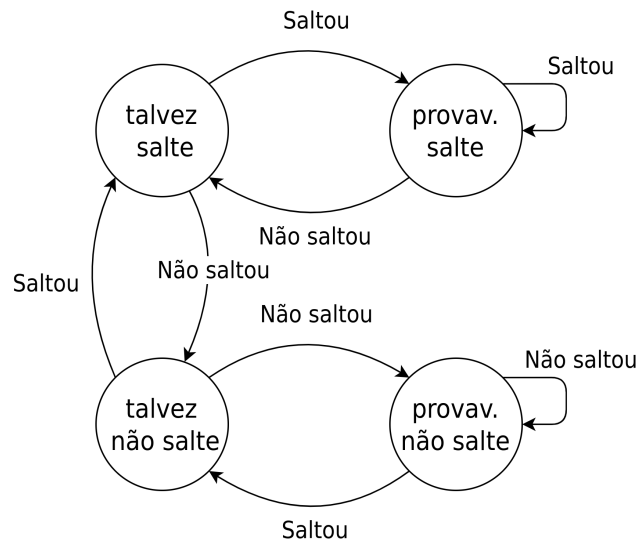


Figura 7: Máquina de estados usada em um preditor de branches bimodal.

3.5 Previsão dinâmica de branches

Se o impacto da correta especulação de condicionais é tão grande em pipelines profundos, técnicas mais avançadas (e custosas em várias métricas) começam a se tornar mais interessantes, para que possamos extrair cada gota adicional de desempenho do nosso processador. Na prática, como a questão é estatística e não-determinística, vamos criar aproximações heurísticas do comportamento futuro – mas elas não precisam ser tão complicadas assim.

Preditor bimodal

Uma maneira simples de melhorar a previsão é simplesmente fazer um circuito adaptativo, ou seja, quando a previsão se mostra errada, trocamos a previsão. Para tanto, podemos fazer uma máquina de estados, vista na figura 7, que descreve qual será nosso chute e o quanto estamos confiantes nele.

Para que isto funcione, vamos manter uma máquina de estados para cada um dos branches condicionais que o processador encontrar durante a execução. Na primeira vez em que encontrarmos esta condicional, vamos chutar com baixa confiança (indicado por “talvez” na nossa máquina de estados), podendo usar o critério anterior da previsão estática. Vamos supor, por exemplo, que seja um salto para trás, o que implica um loop, então chutamos que vai saltar, com baixa confiança.

Na hora de efetivamente executar essa instrução no final do pipeline, verificamos se o nosso chute foi acertado; caso haja o salto, mudamos o es-

tado para alta confiança de que irá saltar novamente na próxima vez em que encontrarmos este branch. Caso *não haja o salto ao final*, ou seja, fizemos a previsão errada, mudamos nosso palpite na máquina de estados para prever que não haverá salto, com baixa confiança; adicionalmente, *precisaremos fazer um pipeline flush* já que erramos nossa previsão e carregamos instruções que não devem ser executadas, perdendo tantos clocks quanto forem os estágios do pipeline.

Os demais casos são análogos: se acertamos, aumentamos nossa confiança; se erramos, sempre fazemos um flush do pipeline e caminhamos na direção oposta ao nosso chute (ou diminuindo a confiança ou trocando de opinião). Os casos em que este esquema tem pior desempenho são aquelas condições que variam muito durante a execução do programa entre verdadeiras ou falsas, tais como algumas verificações de parâmetros em funções. Portanto, a ideia por trás da heurística é que tipicamente as condições de ifs e loops não vão ter muita variação e, mesmo começando com um chute errado, logo vamos convergir para uma condição de certeza (indicada como “provavelmente” na máquina de estados).

Para implementar isso em hardware, precisaremos de uma tabela com duas colunas: a primeira coluna tem os endereços das instruções condicionais na memória de programa e a segunda é uma indicação do estado atual do preditor. Assim, quando encontramos uma condicional no programa, procuramos o endereço dela nesta tabela (se não existir, “criamos” uma nova entrada, talvez descartando uma das linhas presentes) e observamos o estado atual, que pode ser codificado com apenas dois bits. De acordo com ele, vamos chutar o endereço da instrução seguinte e continuar o pipeline normalmente.

Na finalização de uma instrução condicional no pipeline, podemos verificar na tabela se a nossa previsão foi acertada e, em sequência, atualizar o estado de acordo com a máquina de estados vista, fazendo flush do pipeline ou não, de acordo com o erro ou acerto.

Há algumas técnicas sobre esta tabela e sua implementação; mais à frente iremos discutir mecanismos muito similares utilizados no gerenciamento das memórias cache.

Outros preditores

Dado que os pipelines profundos são tão comuns, quaisquer ganhos, mesmo que sejam de décimos percentuais, podem valer a pena, com o trade-off de termos circuitos mais complexos e que consomem mais corrente.

Por exemplo, pode-se implementar uma tabela única mapeando o histórico das n condicionais mais recentes, marcando desvios tomados e não tomados, mas que é utilizada para todos os branches do programa. Cada um dos 2^n históricos possíveis usa um preditor bimodal. A isso chama-se uma *branch history table* global, e é um método alternativo a ter preditores

bimodais individualizados para cada branch. Uma das consequências deste método é que todos os branches compartilham este histórico, portanto ele fica “embaralhado,” por assim dizer, e para tentar prever um único branch ele depende da ordem de execução e dos padrões de desvio de todos os branches.

Talvez surpreendentemente, em algumas situações este método dá resultados *melhores* do que o preditor bimodal. Mas em algumas situações ele dá resultados piores... Mas não precisamos escolher entre os dois: podemos *combinar dois ou mais preditores diferentes de branch*.¹¹ Assim, o processador implementa ambos os preditores (ou mais ainda!) e combina as previsões deles de acordo com algum critério.

No Intel Nehalem, temos uma combinação entre o global, que acerta até 96% das previsões, e o bimodal, que satura no máximo em 93% mas que é melhor do que o global em várias situações.^{12 13} Com a combinação de ambos, esperamos uma taxa de acerto de até 97% das previsões de branches.

Isso sugere que uma mera heurística pode ser melhorada com um trabalho estatístico, portanto não deve ser surpresa que os processadores da AMD utilizem, há algum tempo já, redes neurais artificiais como auxiliares na predição (afinal, um circuito destes é nada mais do que uma tabela do Excel versão deluxe, que realiza uma regressão dos pontos de funcionamento do circuito, de forma adaptativa).

Talvez seja mais uma jogada de marketing do que ser uma técnica com muitos ganhos, mas não temos os números disponíveis para saber com certeza. Tratam-se de poucos neurônios, ocupando pouco espaço e simulados com bastante rapidez por serem implementados diretamente em hardware, e efetivamente isso é um uso de *machine learning* dentro de um processador. Note-se que não é o único uso de ML que encontraremos.

¹¹O artigo que deu origem a esta técnica de combinar preditores e descreve os métodos é o “Combining Branch Predictors” de Scott McFarling, publicado em 1993 pela Western Research Laboratory na Technical Note TN-36. Está disponível em <https://www.hp1.hp.com/techreports/Compaq-DEC/WRL-TN-36.pdf>.

¹²Estes números são relativos a algumas aplicações do benchmark SPEC89 (ibid.).

¹³Existem algumas formas típicas de combinar preditores, nenhuma delas simples o suficiente para explicar neste rodapé. Note também que os percentuais de acerto dependem do tamanho das tabelas envolvidas etc. (ibid.).